

Parameterized Quantum Circuits

QML

Contents

1	What Are Parameterized Quantum Circuits?	3
2	Single Qubit Rotation Gates	3
2.1	$R_x(\theta)$ — Rotation Around X-Axis	3
2.2	$R_y(\theta)$ — Rotation Around Y-Axis	3
2.3	$R_z(\theta)$ — Rotation Around Z-Axis	4
2.4	General Single-Qubit Rotation	4
3	Ansatz Design	4
3.1	What is an Ansatz?	4
3.2	Hardware-Efficient Ansatz (HEA)	4
3.3	Problem-Inspired Ansatzes	5
3.4	Entangling Patterns	5
4	Expressibility of PQCs	6
4.1	What Does Expressibility Mean?	6
4.2	Entangling Capability	6
5	Circuit Depth vs Expressibility Tradeoff	7
6	Data Encoding Circuits	7
6.1	Encoding Strategies	7
6.2	Data Encoding Comparison	9
7	The Full Variational Circuit	9
8	Gradient Computation: The Parameter Shift Rule	9
8.1	The Problem	9
8.2	Derivation	10
8.3	Gradient-Free Methods	10
9	Common PQC Architectures for QML	11
9.1	Circuit 1: RealAmplitudes	11

9.2	Circuit 2: EfficientSU2	11
9.3	Circuit 3: TwoLocal	11
9.4	Circuit 4: Data Re-uploading Classifier	12
10	The Function Space of PQC's	12
10.1	Fourier Analysis of PQC Outputs	12
11	Summary	12

1 What Are Parameterized Quantum Circuits?

Definition 1.1. a parameterized quantum circuit (PQC), also called a variational ansatz, is a quantum circuit containing gates with tunable parameters $\theta = (\theta_1, \theta_2, \dots, \theta_p)$:

$$U(\theta) = U_L(\theta_L) \cdot U_{L-1}(\theta_{L-1}) \cdots U_1(\theta_1)$$

the circuit produces a parameterized quantum state $|\psi(\theta)\rangle = U(\theta)|0\rangle^{\otimes n}$ and the output is typically an expectation value:

$$f(\theta) = \langle 0|^{\otimes n} U^\dagger(\theta) O U(\theta) |0\rangle^{\otimes n} = \langle O \rangle_\theta$$

where O is some observable (e.g., a pauli operator).

PQCs r the workhorses of NISQ quantum computing. they r used in VQE, QAOA, quantum classifiers, quantum generative models — basically everything.

Key Point. the analogy to classical ML:

classical NN	PQC
weights and biases θ	rotation angles θ
linear layer + activation	parameterized gates + entangling gates
forward pass	circuit execution
backprop	parameter shift rule
ReLU, sigmoid	measurement (born rule)
hidden layers	circuit layers (depth)
width (neurons per layer)	qubits

2 Single Qubit Rotation Gates

the building blocks of PQCs r single-qubit rotation gates. these r the parameterized gates that get optimized during training.

2.1 $R_x(\theta)$ — Rotation Around X-Axis

$$R_x(\theta) = e^{-i\theta X/2} = \begin{pmatrix} \cos \frac{\theta}{2} & -i \sin \frac{\theta}{2} \\ -i \sin \frac{\theta}{2} & \cos \frac{\theta}{2} \end{pmatrix}$$

$$|\psi\rangle \text{ --- } \boxed{R_x(\theta)} \text{ ---}$$

effect on the bloch sphere: rotation by angle θ around the x-axis.

special cases: $R_x(0) = I$, $R_x(\pi) = -iX$, $R_x(2\pi) = -I$.

2.2 $R_y(\theta)$ — Rotation Around Y-Axis

$$R_y(\theta) = e^{-i\theta Y/2} = \begin{pmatrix} \cos \frac{\theta}{2} & -\sin \frac{\theta}{2} \\ \sin \frac{\theta}{2} & \cos \frac{\theta}{2} \end{pmatrix}$$

R_y is special bcs its matrix entries r real. this means R_y can create any real superposition: $R_y(\theta)|0\rangle = \cos(\theta/2)|0\rangle + \sin(\theta/2)|1\rangle$.

2.3 $R_z(\theta)$ — Rotation Around Z-Axis

$$R_z(\theta) = e^{-i\theta Z/2} = \begin{pmatrix} e^{-i\theta/2} & 0 \\ 0 & e^{i\theta/2} \end{pmatrix}$$

R_z only changes the relative phase between $|0\rangle$ and $|1\rangle$. it doesnt change the measurement probabilities in the computational basis.

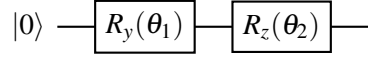
2.4 General Single-Qubit Rotation

any single-qubit unitary can be decomposed as:

$$U(\alpha, \beta, \gamma, \delta) = e^{i\alpha} R_z(\beta) R_y(\gamma) R_z(\delta)$$

so 3 parameters (ignoring global phase) suffice for any single-qubit gate. in PQCs, we typically use combinations of R_y and R_z (or R_x and R_z) to parameterize single-qubit operations.

Example 2.1. a common single-qubit variational layer:



this can produce any state on the bloch sphere (2 parameters for 2 degrees of freedom).

3 Ansatz Design

3.1 What is an Ansatz?

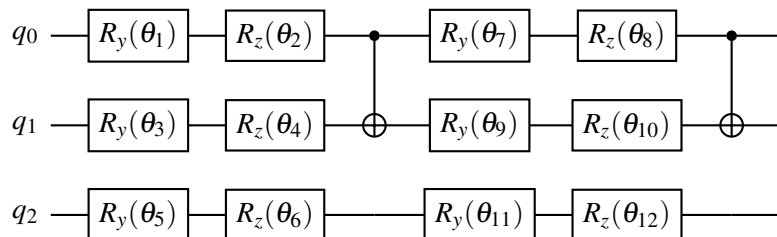
Definition 3.1. an ansatz is the specific circuit architecture u choose for ur PQC. it defines:

1. which parameterized gates to use
2. how to arrange them (layer structure)
3. which qubits to entangle and how
4. the total number of parameters

the choice of ansatz determines the hypothesis class of ur quantum model.

3.2 Hardware-Efficient Ansatz (HEA)

Definition 3.2. a hardware-efficient ansatz is designed to match the native gate set and connectivity of a specific quantum processor. layers alternate between single-qubit rotations and entangling gates native to the hardware.



each “layer” consists of:

1. single-qubit rotations on every qubit (R_y, R_z)
2. entangling gates between connected qubit pairs (CNOT, CZ)

repeat for L layers. total parameters: $\sim 2nL$ (for n qubits).

Key Point. HEA pros:

- low circuit depth (native gates, no compilation overhead)
- matches hardware connectivity (no SWAP gates needed)
- simple to implement

HEA cons:

- no problem structure encoded — the ansatz doesn’t “know” anything about ur task
- prone to barren plateaus (random-looking circuits at initialization)
- may need many layers to be expressive enough

3.3 Problem-Inspired Ansatzes

Definition 3.3. a problem-inspired ansatz uses domain knowledge to design the circuit. examples:

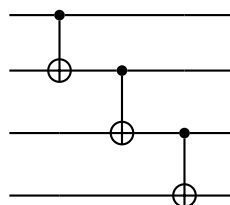
- UCCSD (unitary coupled cluster): for quantum chemistry. encodes the structure of electron excitations
- QAOA: for combinatorial optimization. alternates between problem and mixer hamiltonians
- Hamiltonian variational ansatz: uses trotterized time evolution under the problem hamiltonian

Intuition. problem-inspired ansatzes trade generality for efficiency. by encoding problem structure, they can reach good solutions with fewer parameters and shallower circuits. the downside is u need to know smtg abt ur problem to design the ansatz. for QML, this means choosing the right data encoding and circuit structure for ur specific classification or regression task.

3.4 Entangling Patterns

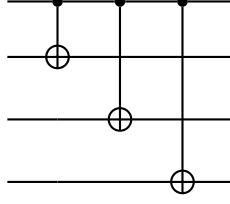
the entangling structure significantly affects the circuit’s expressive power:

linear entanglement: each qubit entangles with its neighbor.



circular entanglement: same as linear but with an additional gate connecting the last qubit to the first.

full (all-to-all) entanglement: every pair of qubits gets an entangling gate.



Key Point. more entanglement = more expressiveness but also more noise and potentially more barren plateaus. the sweet spot depends on the problem and the hardware.

4 Expressibility of PQCs

4.1 What Does Expressibility Mean?

Definition 4.1. the expressibility of a PQC measures how well the circuit can explore the full hilbert space. a highly expressible circuit can produce states close to any target state. mathematically, its quantified by how close the distribution of output states is to the uniform (Haar random) distribution over unitaries.

sim, johnson, and aspuru-guzik (2019) introduced a quantitative measure:

$$\text{Expr} = D_{\text{KL}}(\hat{P}_{\text{PQC}}(F) \| P_{\text{Haar}}(F))$$

where $F = |\langle \psi | \phi \rangle|^2$ is the fidelity between pairs of randomly parameterized states, $\hat{P}_{\text{PQC}}$ is the distribution from the PQC, and P_{Haar} is the distribution from Haar random unitaries.

low expressibility value = the PQC covers the hilbert space well = good.

Example 4.1. expressibility hierarchy (from their paper):

- single R_y layer, no entanglement: low expressibility (restricted to real product states)
- $R_y + R_z$ layers with CNOT entanglement: moderate expressibility
- multiple layers of $R_y + R_z +$ CNOT: high expressibility (approaches Haar random)
- too few parameters or wrong topology: low expressibility regardless of depth

4.2 Entangling Capability

Definition 4.2. the entangling capability of a PQC measures how much entanglement the circuit can generate, averaged over random parameter values. typically measured using the meyer-wallach entanglement measure:

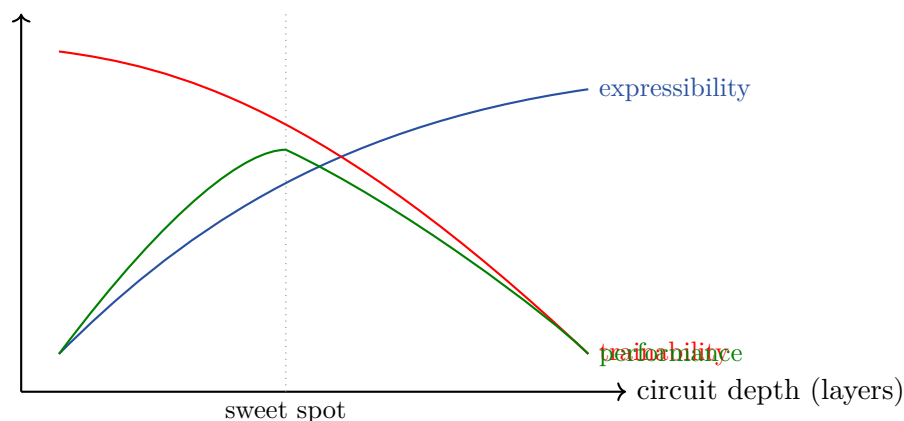
$$\mathcal{Q} = \frac{2}{n} \sum_{j=1}^n (1 - \text{tr}(\rho_j^2))$$

where ρ_j is the reduced density matrix of qubit j . $\mathcal{Q} = 0$ for product states, $\mathcal{Q} \rightarrow 1$ for highly entangled states.

Key Point. expressibility and entangling capability r related but distinct:

- u can have a highly expressible circuit with low entangling capability (e.g., many single-qubit gates, no entanglement)
- u need BOTH expressibility AND entangling capability for powerful QML models
- but too much of either can lead to trainability issues (barren plateaus)

5 Circuit Depth vs Expressibility Tradeoff



Key Point. the depth dilemma:

- too shallow: not expressive enough. the model cant represent the target function. under-fitting.
- too deep: hits barren plateaus (gradients vanish exponentially). also more noise. cant train even if the model is expressive enough.
- sweet spot: enough depth for the task but not so much that training breaks. typically 5–20 layers for current QML tasks.

finding this sweet spot is one of the open challenges in QML.

6 Data Encoding Circuits

in QML, PQCs have two types of parameters:

1. data parameters: encoding the classical input x into the quantum state (the “feature map”)
2. trainable parameters: angles that r optimized during training

6.1 Encoding Strategies

basis encoding: encode x as a computational basis state.

$$x = 5 = 101_2 \rightarrow |101\rangle$$

simple but limited. needs $n = \lceil \log_2 N \rceil$ qubits for N data points.

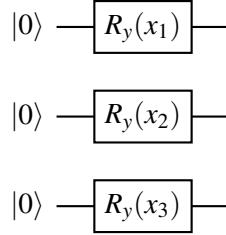
amplitude encoding: encode the data vector as amplitudes of a quantum state.

$$\vec{x} = (x_1, \dots, x_N) \rightarrow |\psi_x\rangle = \frac{1}{\|\vec{x}\|} \sum_{i=1}^N x_i |i\rangle$$

exponentially compact ($\log N$ qubits for N -dimensional data) but hard to prepare efficiently.

angle encoding: encode each feature as a rotation angle.

$$|\psi_x\rangle = \bigotimes_{i=1}^n R_y(x_i) |0\rangle$$



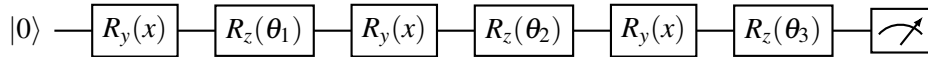
simple and efficient. but product states — no entanglement. limited expressiveness.

IQP (instantaneous quantum polynomial) encoding: uses Z-rotations and entangling gates.

$$U(x) = \exp \left(i \sum_j x_j Z_j + i \sum_{j < k} x_j x_k Z_j Z_k \right)$$

this creates entanglement between features and is used in the Havlicek et al. (2019) quantum kernel classifier.

data re-uploading: encode data multiple times throughout the circuit, interleaved with trainable gates.



Key Point. Perez-Salinas et al. (2020) showed that data re-uploading with a single qubit is a universal quantum classifier. By encoding data multiple times, each layer can implement increasingly complex nonlinear functions of the input. This is analogous to how depth in classical neural networks increases function complexity.

The function computed by a data re-uploading circuit with L layers is a partial Fourier series with frequencies up to L :

$$f(x) = \sum_{k=-L}^L c_k e^{ikx}$$

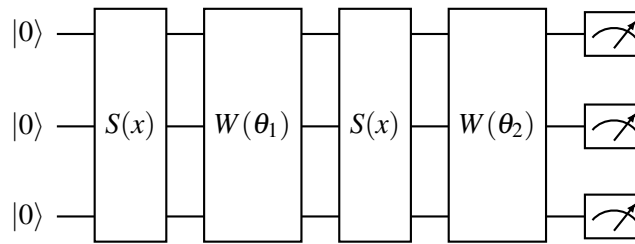
more layers = higher frequencies = more complex decision boundaries.

6.2 Data Encoding Comparison

encoding	qubits for d features	circuit depth	expressiveness
basis	$\lceil \log_2 d \rceil$	$O(d)$	limited
amplitude	$\lceil \log_2 d \rceil$	$O(d)$	limited
angle	d	$O(1)$ per feature	moderate
IQP	d	$O(d)$	high
re-uploading	≥ 1	$O(Ld)$	universal

7 The Full Variational Circuit

a typical variational QML circuit combines data encoding and trainable layers:



where $S(x)$ is the data encoding circuit and $W(\theta)$ is the trainable variational layer.

the full circuit is:

$$U(\theta, x) = W(\theta_L)S(x)W(\theta_{L-1})S(x) \cdots W(\theta_1)S(x)$$

and the model output is:

$$f(\theta, x) = \text{tr}(O \cdot U(\theta, x)|0\rangle\langle 0|U^\dagger(\theta, x))$$

Key Point. the structure matters enormously:

- wt data encoding $S(x)$ u use determines the quantum feature space
- wt trainable circuit $W(\theta)$ u use determines wt functions u can learn within that feature space
- the number of re-uploads of the data determines the frequency spectrum of the model
- the entanglement structure determines correlations between features

this is not just hyperparameter tuning — the circuit architecture fundamentally determines wt the model can and cannot learn.

8 Gradient Computation: The Parameter Shift Rule

8.1 The Problem

to train a PQC, we need gradients $\frac{\partial f}{\partial \theta_i}$. but we cant do backprop on a quantum circuit (no intermediate states r accessible). the solution: the parameter shift rule.

8.2 Derivation

Definition 8.1. for a gate of the form $G(\theta) = e^{-i\theta P/2}$ where P is a pauli operator ($P^2 = I$), the gradient is:

$$\boxed{\frac{\partial}{\partial \theta} \langle O \rangle_\theta = \frac{\langle O \rangle_{\theta+\pi/2} - \langle O \rangle_{\theta-\pi/2}}{2}}$$

derivation: the expectation value is:

$$\langle O \rangle_\theta = \langle 0 | U_-^\dagger(\theta) G^\dagger(\theta) U_+^\dagger O U_+ G(\theta) U_-(\theta) | 0 \rangle$$

where U_+ and U_- are the circuit parts after and before the gate $G(\theta)$.

using $G(\theta) = \cos(\theta/2)I - i\sin(\theta/2)P$ and differentiating:

$$\frac{\partial G}{\partial \theta} = -\frac{1}{2}\sin(\theta/2)I - \frac{i}{2}\cos(\theta/2)P \quad (1)$$

$$= -\frac{i}{2}P \cdot G(\theta) = \frac{1}{2}(G(\theta + \pi/2) - G(\theta - \pi/2)) \cdot (\text{using } e^{-iP\pi/4}) \quad (2)$$

after working through the algebra:

$$\frac{\partial \langle O \rangle}{\partial \theta} = \frac{1}{2}(\langle O \rangle_{\theta+\pi/2} - \langle O \rangle_{\theta-\pi/2})$$

Key Point. the parameter shift rule is beautiful bcs:

- it gives the exact gradient (not an approximation like finite differences)
- it only requires 2 circuit evaluations per parameter
- it works on real quantum hardware (no need for intermediate state access)
- total cost for all p gradients: $2p$ circuit evaluations
- this is worse than classical backprop ($O(1)$ overhead) but much better than finite differences ($O(p)$ with approximation error)

Example 8.1. compute $\frac{\partial}{\partial \theta} \langle Z \rangle$ for the circuit $R_y(\theta) | 0 \rangle$:

$$\langle Z \rangle_\theta = \cos \theta$$

$$\text{parameter shift: } \frac{1}{2}(\cos(\theta + \pi/2) - \cos(\theta - \pi/2)) = \frac{1}{2}(-\sin \theta - \sin \theta) = -\sin \theta$$

$$\text{direct differentiation: } \frac{d}{d\theta} \cos \theta = -\sin \theta. \quad \checkmark$$

8.3 Gradient-Free Methods

alternatives to gradient-based optimization:

- COBYLA: constrained optimization by linear approximation. no gradients needed.
- SPSA (simultaneous perturbation stochastic approximation): estimate the gradient with only 2 circuit evaluations total (independent of p). noisy but cheap.
- Nelder-Mead: simplex method. good for low-dimensional optimization.

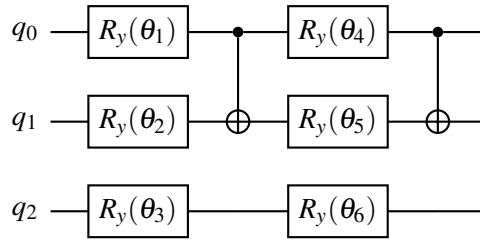
- Bayesian optimization: builds a surrogate model of the loss landscape. good for expensive evaluations.

Key Point. in practice, the choice between gradient-based and gradient-free optimization depends on:

- number of parameters (gradient-free methods don't scale well beyond ~ 50 – 100 parameters)
- noise level (noisy gradients may make gradient-free methods preferable)
- circuit evaluation cost (if each circuit run is expensive, SPSA with 2 evaluations per step is attractive)
- the loss landscape (if it's very noisy or has many local minima, gradient-free methods may navigate better)

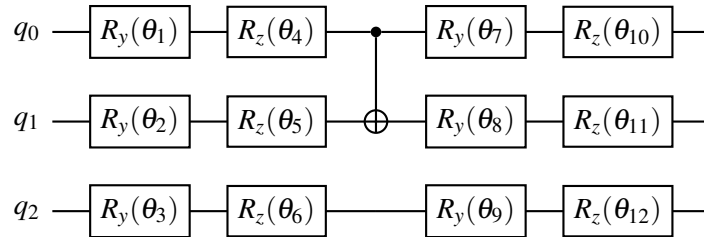
9 Common PQC Architectures for QML

9.1 Circuit 1: RealAmplitudes



only R_y rotations (real amplitudes) with linear CNOT entanglement. simple, low parameter count, easy to train.

9.2 Circuit 2: EfficientSU2

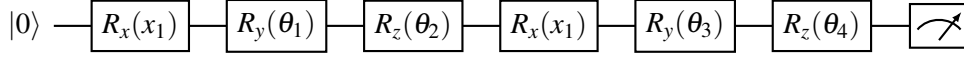


$R_y + R_z$ rotations (full $SU(2)$ on each qubit) with entanglement. more expressive than RealAmplitudes.

9.3 Circuit 3: TwoLocal

a generalization where u can choose any rotation gates and any entanglement pattern. the qiskit `TwoLocal` class lets u specify both.

9.4 Circuit 4: Data Re-uploading Classifier



single qubit, data encoded multiple times. universal for any number of features (with enough qubits and layers).

10 The Function Space of PQCs

10.1 Fourier Analysis of PQC Outputs

Key Point. schuld, sweke, and meyer (2021) showed that the functions computed by PQCs are partial fourier series. specifically, for a PQC with data encoding gates $e^{-ix_j G_j}$ (where G_j are generators with eigenvalues in a discrete set Λ), the output function is:

$$f(x) = \sum_{\omega \in \Omega} c_{\omega} e^{i\omega x}$$

where Ω is the set of accessible frequencies, determined by the eigenvalues of the generators and the number of times data is encoded.

key implications:

- the data encoding determines which frequencies are available
- the trainable parameters determine the fourier coefficients c_{ω}
- more data re-uploads = higher frequencies = more complex functions
- this gives a precise characterization of what functions PQCs can and cannot learn

Example 10.1. single qubit, $R_x(x)$ encoding, L re-uploads:

accessible frequencies: $\omega \in \{-L, -L+1, \dots, L-1, L\}$

the model can represent $2L+1$ fourier components $\rightarrow$ functions with resolution $\sim 1/L$.

11 Summary

Key Point. PQCs in a nutshell:

1. PQCs are quantum circuits with tunable parameters — the quantum analogue of neural networks
2. rotation gates (R_x, R_y, R_z) provide the parameterization
3. ansatz design (hardware-efficient vs problem-inspired) determines the hypothesis class
4. expressibility measures how well the ansatz covers the hilbert space
5. data encoding (angle, amplitude, IQP, re-uploading) determines the quantum feature space
6. the parameter shift rule gives exact gradients with $2p$ circuit evaluations

7. PQC outputs r partial fourier series — the encoding determines frequencies, the trainable parameters determine coefficients
8. the sweet spot between expressibility and trainability is the central design challenge

next: QAOA — the first major application of PQCs to combinatorial optimization, and a stepping stone to quantum classifiers.